

朱银杭-3 年-简历

基本信息	
姓 名：朱银杭	毕业院校：南开大学
性 别：男	学 历：本科
年 龄：25	工作时间：三年
电 话：131-6991-9816	邮 箱：616598128@qq.com
现 居 地：深圳市	籍 贯：贵州省
求职意向	

期望职位：Java 工程师

期望地点：深圳

期望薪资：16 ~ 18k

技术能力

- 熟练掌握 Java 基础，了解 JVM 内存结构；
- Docker 容器，RabbitMQ 的开发使用经验；
- 熟练掌握 Mybatis, SpringBoot 等主流框架的使用了解 SpringCloud 及其主要组件；
- 有微服务的实际开发经验，会使用 nacos 和 compose 优化配置；
- 熟悉 MySQL，Redis 缓存中间件的使用，熟悉基本 Linux 命令；
- 熟悉 Java8 新特性，会使用 Lambda 表达式结合 stream 流，Mybatis-plus 高效编码，代码符合阿里开发规范，保证可读性；
- 熟悉 Git、Maven、IDEA、Jenkins、postman、禅道等工具的使用；

工作经历

公司名称 广州菁人教育

所属行业 教育培训

所属部门 数学教研组

职位名称 高中数学教师

在职时间 2018-12 至 2020-07

工作内容

1. 制定课时计划
2. 编教案，上课
3. 培训新老师

公司名称 高伟达

所属行业 互联网

所属部门 存房研发组

职位名称 Java 后端工程师

在职时间 2020-12 至今

工作内容

1. 评审业务流程
2. 完成模块开发
3. 处理 bug
4. 辅助架构师处理环境漏洞与版本升级

项目经验

建行旗下的房屋租赁系统

分存房，长租，短租，监管，开放平台等项目组

本人在存房项目组担任后端开发，项目组后端 8 人，前端 3 人，运维一人，架构一人，dba 一人，产品经理一人，项目经理一人

存房系统基于 springcloud 脚手架开发，供全国的总，分子公司，运营商，服务商内部员工使用。

前后端分离，后端 Java，前端 vue，共开发，测试，业务测试，生产四个环境，nacos 管理配置，服务，jenkins 统一管理服务部署与上线流水线打包。

存房共系统，工作流，房源，合同，资产，出租，装配，交割，报表等十多个模块，

feign 接口调用同项目模块，httpclient 走其他项目的接口。

我主要负责系统用户权限，房屋交割，房源发布等模块，和一些卸数装数流程。

mysql(主要使用)，pg，oracle 都有用到，优化流程时一般用 rabbit 发两条异步消息，一条给数据库做事务，一条推送给其他模块相关信息。

独立开发集中式存入(企业房源)的筹开模块，接受项目时的信息(其中有存入签约，交割，装配，运营商签约等一系列的逻辑判断)，录入运营商提供的材料并与租赁系统对接数据，扭转到下一流程。

工作流程

代码管理

存房代码托管于 icdp 上的 git, 代码下载地址如下, 先进入 icdp 主页, 找到如下菜单, icdp 网址为 `icdp.dev.jh/cs_f/#/code/manage`

选择对应的项目

存房采用微服务的形式, 每个微服务都有单独的项目工程, 目前有以下工程:

工作台服务 (desk)

申请服务 (apply): 分散式申请、集中式项目管理

房源服务 (house): 房源、权属、网点核验

合同服务 (contract): 合同、收房方案

交割服务 (delivery): 交割

出租服务 (rent): 运营商委托、出租管理 (房源发布)、定价管理

投资服务 (invest): 旧版投资、装配

资产服务 (asset): 资产

报表服务 (report): 报表

财务服务 (finance): 财务管理

系统服务 (system): 系统设置

文件服务 (file): 文件上传相关

网关服务 (gateway): 网关

工作流服务 (workflow): 工作流相关

其中, 系统服务、文件服务、网关服务、工作流服务共用一个工程 (basic)

分支管理

每个项目工程都有三个基本分支, 即开发 (develop)、测试 (test) 和主 (master) 分支。开发人员开发必须在 develop 分支进行, 开发完成后经自测 (可用 postman 自测) 没问题时, 将代码提交, 然后在开发服务器上启动该服务, 启动完成后进入开发环境的存房页面, 确定功能没问题后, 将 develop 分支的代码合并至 test 分支, 然后通知测试人员 (目前是我们公司的阳艳) 启动对应的测试服务并提醒她测试该功能。如果测试发现有 bug, 则需要开发人员在 develop 分支进行修改, 然后再重复上面的步骤……经测试人员测试没问题时, 等待业务测试环境提测日, 到达那天时, 将 test 分支的代码合并至 master 分支, 同时通知运维人员 (目前是彭启志) 启动业务测试环境的对应服务 (我们没有权限启动)。

环境管理

目前存房有三个环境, 分别是:

开发环境: 128.195.26.106 测试环境: 128.195.26.107 业务测试环境: 128.195.26.108

其中业务测试环境为准生产环境。

maven 配置

采用行内提供的 maven 仓库, 具体 settings.xml 文件内容 (简历上不展示)

服务管理

存房的各个微服务是运行于对应环境的 docker 容器内的, 平常自己本地开发时, 不需要运行在 docker 容器内, 只需像 spring boot 启动一样, main 方法运行。

微服务之间的调用采用 feign 的形式 (**注意: 本地开发时无法调试微服务的调用, 因为被调用的微服务存在于开发环境的 docker 容器内**)。

服务启动

开发和测试环境的微服务通过 jenkins 启动，jenkins 的配置由运维完成。Jenkins 的地址为 128.195.26.132:8000。其中的“dev”和“test”分别是开发和测试环境的服务所在地，点击以下按钮可启动对应的服务：

all-环境相关

dev-业务服务

dev-前端

dev-基础服务

test-业务服务

test-前端

test-基础服务

uat-业务服务

uat-前端

uat-基础服务

所有

S	W	名称 ↓	上次成功	上次失败	上次持续时间	收藏
		dev-apply	6 小时 38 分 - 20200325_044826	9 小时 37 分 - 20200325_014545	7 分 35 秒	
		dev-asset	4 天 16 小时 - 20200320_190837	7 天 11 小时 - #27	43 分	
		dev-checkout	1 天 12 小时 - 20200323_223002	7 天 11 小时 - #64	3 分 43 秒	
		dev-contract	17 小时 - 20200324_174228	7 天 11 小时 - #66	4 分 18 秒	
		dev-delivery	1 天 20 小时 - 20200323_142356	7 天 11 小时 - #106	5 分 59 秒	
		dev-desk	4 天 16 小时 - 20200320_185235	7 天 11 小时 - #28	27 分	

开发人员只需要启动“dev-业务服务”中的服务，测试环境由测试人员启动。

“uat”为业务测试环境，因为业务测试环境启动服务是在 idcp 上进行的，这里的“uat”只用于服务启动的查看，即 idcp 上启动完成时，jenkins 上的“上次成功”时间会刷新。

日志查看

有时候遇到问题，会需要查看日志，各个环境查看日志方法如下：

(1) 开发环境

用 Linux 远程连接工具（云桌面已自带连接工具：桌面→常用软件→SecureCRT 6.6）登录开发服务器（端口：22，用户名：hess，密码：cunfang123）。

输入以下命令查看日志（开发环境只能通过 docker 查看日志）

dev, aut 环境：docker logs -f -tail 200 服务名
test 环境：docker service logs -f -tail 200 hess-test_服务名

其中，-f 表示查看实时日志，--tail 200 表示查看最后 200 行日志，服务名为对应微服务的英文名（省略 hydosky-前缀）。

其他的日志查看命令还有：

docker logs -since 2020-03-16T12:00:00 服务名
查看某个服务在 2020-03-16 12:00:00 之后的日志
docker logs -until 2020-03-16T12:00:00 服务名
查看某个服务在 2020-03-16 12:00:00 之前的日志

(2) 测试和业务测试环境

登录测试环境（端口：22，用户名：hess，密码：hess_1234）。

测试和业务测试环境有两种查看日志的方法，第一种跟开发环境一样，第二种为查看日志文件，下面说一下第二种：

在用户根目录（登录服务器之后进入的目录就是用户根目录）中输入以下命令：

cd log/hydosky-服务名
ls

找到其中两个文件 info.log 和 error.log，这两个即为此时的日志文件，查看该文件可看到对应的日志。

公共配置（数据库链接、redis 等）查看

为方便统一管理，各个微服务之间的公共配置（数据库链接、Redis 配置等等）由 nacos 进行管理，具体，查看方法如下（以开发环境为例，其他环境类似）：

浏览器输入地址：128.195.26.106:8848/nacos（其他两个环境的地址类似，只是 ip 换一下），输入账号、密码，进入以下页面：

点击“详情”，弹出的页面即为公共配置

已经启动的服务列表也可在 nacos 上查看（本地启动的也会在上面显示），

定时任务管理

存房的定时任务使用“xxl-job”，代码书写如下

```
@Slf4j
@Component
public class ApplyHandler {

    @Autowired
    private IApplyInfoService iApplyInfoService;

    @XxlJob(value = "applyStatusExecute")
    public ReturnT<String> execute(String s) {
        log.info("执行：-----");
        boolean result = iApplyInfoService.updateApplyStatus();
        log.info("执行结果：-----" + result);
        return result == true ? ReturnT.SUCCESS : ReturnT.FAIL;
    }
}
```

写好之后，进入 xxl-job 管理页面，网址为 128.195.26.106:8081/xxl-job-admin（其他两个环境类似，只是 ip 换一下）。

点击左侧的“执行器”管理，配置执行器（一个微服务只需要一个）

然后点击左侧的“任务管理”，在右侧页面中选择对应的执行器

点击右侧页面上的“新增”，填写对应的项，点击保存。

数据库脚本变更

如果某次上线版本需要修改数据库，则需要将对应的数据库脚本提交到 Script 项目中
脚本存放的路径如下：sqlfile/\${yyyy-MM-dd}/模块名

其中，\${yyyy-MM-dd} 为上线版本日期，脚本文件命名规范为：模块名_上线版本日期_ddl/dml。

开发环境可自行变更数据库，测试环境需要变更数据库时，通知测试人员到以上路径找到对应的脚本，然后由测试人员将脚本内容放至 yearning 平台进行审核并执行，审核不通过时，需要调整脚本直至审核通过。业务测试环境同理，但是是由运维执行（**这里要注意，开发和测试环境的数据库没有分库，业务测试和生产环境分库了，即一个微服务对应一个数据库**）。

Rabbitmq

报表模块做报表时需要从其他各个服务获取业务数据，目前新的报表采用 rabbitmq 推送的形式，即由对应的模块将报表需要的业务数据推送到 rabbitmq 队列上，报表服务从队列中读取对应的业务数据，rabbitmq 的网址为：128.195.26.106:15672，账号和密码都为 guest。

主要负责的业务

一、用户配置：

1. 所在模块 `system`（系统模块，`HESS_BASIC` 中）
2. 相关表（`hydosky_user`, `hydosky_role`, `hydosky_role_menu`）

（1）新增用户：通过调用企租接口，将用户电话（判重依据）等相关信息存入企租的用户表中。

（2）用户配置：

① 调用企租接口，查询已有的用户，过滤掉存房已配置的角色（`hydosky_user` 表里有的角色），然后返回用户列表；

② 从返回的用户列表里（根据当前登录角色的权限返回可供配置的用户）选择欲配置的用户，配置相应的角色（角色列表从 `hydosky_role` 查出，并根据用户角色作过滤），区域门店等相关信息。

③ 配置的信息提交后，会将相关用户信息存入 `hydosky_user` 里，将用户角色存入 `hydosky_role_menu` 里，并调用企租接口更新职能范围（存放和企租的相同角色配置的角色 `id` 可能不同，有一个关系映射，详情可以查看代码），调用企租接口更新区域门店的配置信息。

④ 用户删除：目前存房和企租的删除是双向联动的。删除存房的用户配置就会删除长租的角色信息，删除长租的角色信息也会删除存放的角色配置。

⑤ 其他模块的角色权限控制都是基于这个逻辑。目前（20211105 版本）共 51 个角色，角色 `id` 从 1（超级管理员）到 51（财务复审）。接口鉴权可以用项目封装的 `@Hydosky-Auth` 注解，也可以在业务代码里 `split` 一下前端传来的类似“11, 35, 32”的角色 `id` 串，转数组后用 `stream` 流配合 `lambda` 表达式作条件过滤有相应权限的用户。

二、集中式项目筹开：

1. 所在模块：**rent** 模块

2. 相关表：**assembleProjectPlan**

3. 新增筹开信息：**集中式**项目信息存储在企租中，**集中式**项目签约后，**工作流**会推筹开。（之前两权，重资产项目必须签约运营商，目前两权，重资产，**受托**的新增逻辑已一致）

（1）有装配

① 如果没有委托运营商，则直接在项目筹开表生成一条待筹开的逻辑删除数据，待装配完成后，会将逻辑删除的 **is_deleted** 由 **1 (true)** 改为 **0(false)**;

② 如果有委托运营商，则在运营商签约表生成一条待运营商签约数据，待运营商签约并且装配完成后，再在项目筹开表生成一条待筹开的筹开数据。

（2）无装配

① 如果没有委托运营商，则直接在项目筹开表生成一条待筹开的数据;

② 如果有委托运营商，则在运营商签约表生成一条待运营商签约数据，待运营商签约后，再在项目筹开表生成一条待筹开的筹开数据。

4. 提交筹开材料，变更筹开状态，由待筹开更新为待开业。

（1）调企租接口获取项目**交割**状态，必须**物业交割**完成后才能**录入**筹开信息

（2）预计房源（间数，套数）与实际房源（间数，套数）的差异如果超过 **10%**，需要总公司角色进行审核后才能继续**流程**。

（3）调用企租接口获取出系统已上传房源（间数，套数）

（4）预计房源（间数，套数）与系统已上传房源（间数，套数）的差异如果超过 **10%**，需要总公司角色进行审核后才能继续**流程**。

（5）调企租接口更新项目绑定（绑定出租项目，出租品牌）并更新项目信息

（6）更新筹开信息，状态为待开业

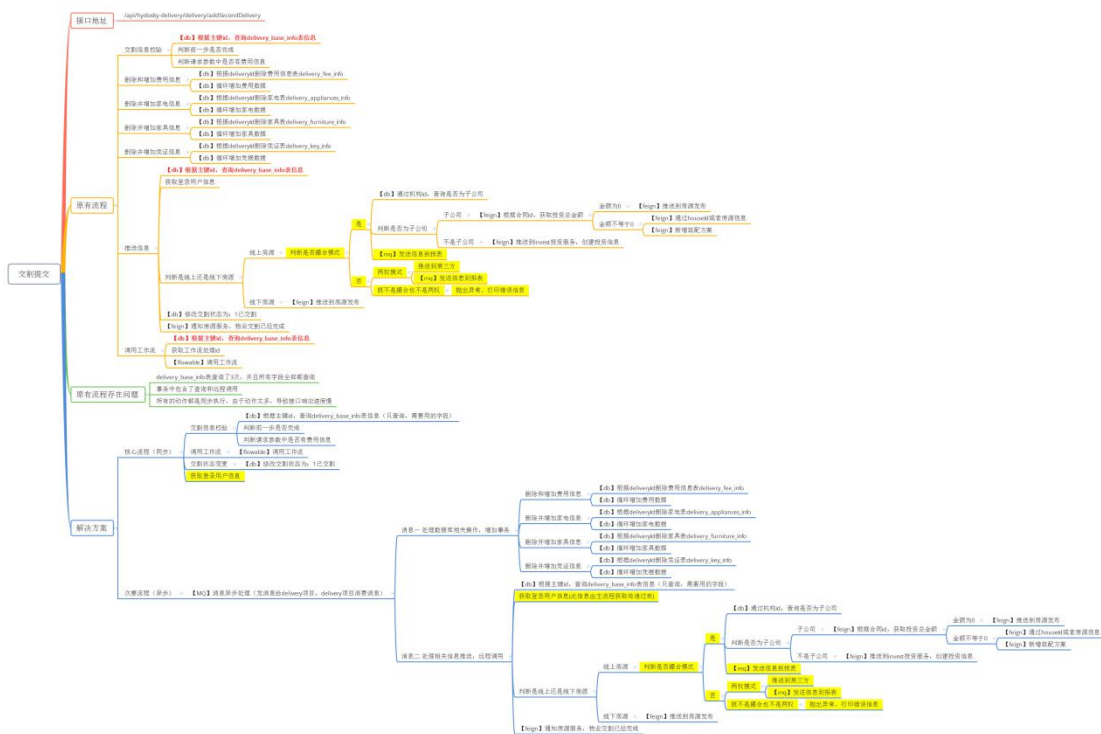
（7）插入操作记录

注：

四、物业交割优化

1. 相关表: `delivery_base` info, `delivery_fee` info,

2.3.1.delivery_appliances_info, delivery_furniture_info, delivery_key_info



五、业财报表

1. 所在模块: finance 模块

2. 相关表（分散式相关初始数据跑批，分页列表，详情列表，模板下载，数据导入，

页面导出,导入审核):business_finance_statistics,business_finance_statistics_disp

业务流程				
流程	每月录入	数据审核	报表生成	报表下载
	<pre>graph LR; subgraph "每月录入"; direction TB; A[分散式导入]; B[集中式导入]; end; A --> C{审核}; B --> C; C -- 通过 --> D[跑批处理生成报表]; C -- 不通过 --> C; D --> E[筛选条件下载报表];</pre>			
角色	分子公司财务岗	分子公司初审岗		分子公司财务、 初审、终审岗、 总部管理员

3. 分散式初始数据跑批（一个房源对应一个收房合同对应多个出租合同）：

（1）获取跑批两权企业（从企业参数表里拿子公司信息）

（2）保存总表信息到 `business_finance_statistics`

（3）获取刚才插入的总表 id 作为明细表 `business_finance_statistics_disp` 的 `statisticId`

（4）遍历企业列表；

① 用企业 id 从合同模块的 `contract_Info` 表里拿有效的收房合同集合（`pageSize = 500`）

② `stream` 流获取合同 id 集合，传到 `invest` 模块，到装配表里拿到装修结算金额和结算日期，用 `contract_id` 做键，封装返回信息做值，流收集为 `map`，配置策略（A，B）
->A

③ `stream` 流获取房源 id 集合，传到 `house` 模块，到房源信息表里拿到流程状态，房间数，用 `house_id` 做键，封装返回信息做值，流收集为 `map`，配置策略（A，B）->A

④ 遍历收房合同，根据 `contract_id`，`house_id` 填充拿到的存房信息

⑤ 传房源 id 集合到 `rent` 模块的房源发布获取长租房间 id，门店 id，门店名

⑥ 获取所有的长租房 id（`rent` 模块有返回对象的具体封装过程）

⑦ 传长租房 id 从长租获取出租合同的起始资金和月租金

⑧ 遍历收房合同，出租信息填充

⑨ 设置明细表属性值

⑩ 批量保存

六、16 张业务源表卸数：

1. 所在模块：`report`

2. 卸数工具写在 `report`，`UploadUtil`

3. 逻辑：`report` 每天跑定时任务，将数据写出 `docker` 容器，放入约定的卸数文件夹

中

4. 配置单独的 feign 超时时间, apply,contract,rent,house,invest,report(原已配置), delivery7 个模块的 docker, compose 编排文件添加路径映射, 将卸数文件写出, 供其他系统调取装数;

5. report 调同步的 feign 接口, 依次通知各模块卸数, 卸数时, 调用卸数工具, 传入卸数 entity 和卸数 service (代码生成器在 basic 模块的 ops 中)。

6. 卸数工具使用了反射机制, 因为 entity 里的属性是 private 的, 在获取声明字段后,

(1) 要设置 attributeAble, 不然无法调用 getter 方法赋值;

(2) 序列号和数据中的换行, 蓝海市的数据需要过滤掉

(3) entity 里的 LocalDateTime 和 LocalDate 要转换为 Date

7. 每个文件不能超过 20 万, 每个文件包含 .dat,.dat.ok,.xml..xml.ok 四个文件

8. 默认每天按上一天的最后修改时间差异卸数

七、楼盘评估

- 1. 所在模块：apply
- 2. 相关表：rent_project_judge_record

业务流程（新增查询）				
	前端请求	后端调用	数据处理	请求返回
流程	参数录入 发送请求 接口调用 数据保存 请求返回 评估数据			
角色	现阶段：建信住房总公司、分子公司所属机构下人员可用 后期规划：对平台中所有入驻企业开放			

- （1）页面输入请求参数，存房后台接受到请求后进行条件封装，调用数据服务接口
- （2）如果请求到有效数据，将查询参数和请求到的数据插入到存房长租公寓评级表中，

新增查询记录

- （3）将查询结果返回给前端

3. 历史查询

- （1）根据条件（查询时间，城市，项目等）筛选查询记录，得到查询列表
- （2）选择需要查看的历史查询，前端将对应的查询 id 发送请求到后台
- （3）查询存房长租公寓评级表，将查询结果封装后，返回给前端

自我评价

- 1. 参与过多个项目的开发，熟悉常见业务处理，能快速融入团队；（团队合作）

2. 逻辑思维能力强，思路清楚，学习能力强，对新技术有强烈的好奇心；（技术）
3. 对工作尽职尽责，乐于从事有挑战性的工作；（态度）
4. 乐于与用户以及同事和领导沟通，以便快速解决项目遇到的问题；（沟通）
5. 具有良好的英语阅读能力，能阅读英文资料、技术文档等；（英语）
6. 单身（婚姻状况）；
7. 喜欢网球，斯诺克，K 歌；无不良嗜好；（爱好）